

FAR AWAY

Personalized Learning & Skill Competition Platform

Project	Far Away - Adaptive Learning Platform
Type	Full-Stack Web Application
Status	POC - 3 Day Sprint
Date	June 10, 2026
Architecture	MERN Stack (MongoDB, Express, React, Node.js)

Key Features

- Personalized Assessment with Adaptive Difficulty
- SkillSwap Peer Learning Network with Leaderboards
- Coin Economy & Reward System
- Weekly Individual & Group Competitions
- Teacher/Mentor Evaluation System
- Real-time Progress Tracking & Analytics

Technology Stack & Architecture

Frontend

Technology	Purpose	Version
React 18	UI Framework	v18+
Next.js 14	Full-stack framework	v14+
TypeScript	Type Safety	v5+
Tailwind CSS	Styling & Utility Classes	v3.3+
Redux Toolkit	State Management	v1.9+
TanStack Query	Server State Management	v5+
Framer Motion	Animations	v10+
Recharts	Data Visualization	v2+
React Hook Form	Form Management	v7+

Backend

Technology	Purpose	Version
Node.js	Runtime	v18+
Express.js	Web Framework	v4.18+
MongoDB	NoSQL Database	v6+
Mongoose	ODM	v7+
JWT	Authentication	jsonwebtoken v9+
Bcrypt	Password Hashing	v5+
Nodemailer	Email Service	v6+
Redis	Caching & Sessions	v6+
Socket.io	Real-time Features	v4+

DevOps & External Services

Service	Purpose
Docker & Docker Compose	Containerization

JWT + Refresh Tokens	Authentication
Nodemailer + SMTP	Email Notifications
Socket.io	Real-time Notifications
AWS S3 / Cloudinary	File Storage
MongoDB Atlas	Cloud Database
GitHub Actions	CI/CD Pipeline

Project Folder Structure

```
far-away/
├── backend/ # Express.js Backend
│   ├── config/ # Configuration files
│   │   ├── database.js # MongoDB connection
│   │   ├── redis.js # Redis configuration
│   │   └── env.js # Environment variables
│   ├── models/ # Database Models
│   │   ├── User.js # User schema
│   │   ├── Assessment.js # Assessment & Questions
│   │   ├── UserProgress.js # User Progress tracking
│   │   ├── Competition.js # Competition models
│   │   ├── LeaderBoard.js # Leaderboard entries
│   │   ├── SkillSwap.js # Mentorship requests
│   │   ├── Coin.js # Coin transactions
│   │   ├── Submission.js # Assessment submissions
│   │   └── Analytics.js # Performance analytics
│   ├── routes/ # API Routes
│   │   ├── auth.js # Authentication routes
│   │   ├── assessments.js # Assessment endpoints
│   │   ├── competitions.js # Competition endpoints
│   │   ├── leaderboard.js # Leaderboard endpoints
│   │   ├── skillswap.js # Mentorship endpoints
│   │   ├── coins.js # Coin system endpoints
│   │   ├── users.js # User profile endpoints
│   │   └── analytics.js # Analytics endpoints
│   ├── controllers/ # Business Logic
│   │   ├── authController.js
│   │   ├── assessmentController.js
│   │   ├── competitionController.js
│   │   ├── leaderboardController.js
│   │   ├── skillswapController.js
│   │   ├── coinController.js
│   │   ├── analyticsController.js
│   │   └── userController.js
│   ├── middleware/ # Custom Middleware
│   │   ├── authMiddleware.js # JWT verification
│   │   ├── roleMiddleware.js # Role-based access control
│   │   ├── errorHandler.js # Global error handling
│   │   ├── validateRequest.js # Request validation
│   │   ├── rateLimiter.js # Rate limiting
│   │   └── logger.js # Request logging
│   ├── validators/ # Schema Validators
│   │   ├── authValidator.js # Auth validation rules
│   │   ├── assessmentValidator.js # Assessment rules
│   │   ├── competitionValidator.js # Competition rules
│   │   ├── userValidator.js # User profile rules
│   │   └── commonValidator.js # Reusable validators
│   └── services/ # Business Services
│       ├── authService.js # Auth logic
│       ├── assessmentService.js # Assessment logic
│       ├── competitionService.js # Competition logic
│       ├── leaderboardService.js # Leaderboard ranking
│       ├── coinService.js # Coin transactions
│       ├── emailService.js # Email notifications
│       └── analyticsService.js # Analytics computation
```

```
■ ■ ■■ adaptiveService.js # Adaptive difficulty logic
■ ■■ utils/ # Helper Functions
■ ■ ■■ errorCodes.js # Error constants
■ ■ ■■ responseHandler.js # Standard response format
■ ■ ■■ jwtUtils.js # JWT utilities
■ ■ ■■ socketEvents.js # Socket.io events
■ ■ ■■ helpers.js # General helpers
■ ■■ sockets/ # Real-time Features
■ ■ ■■ notificationSocket.js # Notifications
■ ■ ■■ liveLeaderboard.js # Live leaderboard updates
■ ■ ■■ competitionSocket.js # Live competition updates
■ ■■ app.js # Express app setup
■ ■■ server.js # Server entry point
■ ■■ .env.example # Environment variables
■ ■■ package.json
■ ■■ Dockerfile
■
■■■ frontend/ # Next.js Frontend
■ ■■ app/ # Next.js 14 app directory
■ ■ ■■ (auth)/ # Auth pages group
■ ■ ■■ login/page.tsx
■ ■ ■■ register/page.tsx
■ ■ ■■ layout.tsx
■ ■ ■■ (dashboard)/ # Protected pages group
■ ■ ■■ dashboard/page.tsx # Main dashboard
■ ■ ■■ assessments/page.tsx # Assessment list
■ ■ ■■ [id]/ # Assessment detail
■ ■ ■■ leaderboard/page.tsx # Leaderboard view
■ ■ ■■ competitions/page.tsx # Competitions
■ ■ ■■ skillswap/page.tsx # Mentorship
■ ■ ■■ profile/page.tsx # User profile
■ ■ ■■ coins/page.tsx # Coin history
■ ■ ■■ analytics/page.tsx # Analytics
■ ■ ■■ layout.tsx # Dashboard layout
■ ■ ■■ api/ # API routes (optional)
■ ■ ■■ layout.tsx # Root layout
■ ■ ■■ page.tsx # Home page
■ ■■ components/ # Reusable Components
■ ■ ■■ auth/ # Auth components
■ ■ ■■ LoginForm.tsx
■ ■ ■■ RegisterForm.tsx
■ ■ ■■ OTPVerification.tsx
■ ■ ■■ assessment/ # Assessment components
■ ■ ■■ QuestionCard.tsx
■ ■ ■■ CodeEditor.tsx
■ ■ ■■ Timer.tsx
■ ■ ■■ SubmitButton.tsx
■ ■ ■■ competition/ # Competition components
■ ■ ■■ CompetitionCard.tsx
■ ■ ■■ TeamForm.tsx
■ ■ ■■ RankingTable.tsx
■ ■ ■■ common/ # Common components
■ ■ ■■ Navbar.tsx
■ ■ ■■ Sidebar.tsx
■ ■ ■■ Card.tsx
■ ■ ■■ Button.tsx
■ ■ ■■ Modal.tsx
■ ■ ■■ LoadingSpinner.tsx
```

```
■ ■ ■ ■ ■ Toast.tsx
■ ■ ■ ■ ■ leaderboard/ # Leaderboard components
■ ■ ■ ■ ■ LeaderboardTable.tsx
■ ■ ■ ■ ■ UserRankCard.tsx
■ ■ ■ ■ ■ analytics/ # Analytics components
■ ■ ■ ■ ■ ProgressChart.tsx
■ ■ ■ ■ ■ SkillBreakdown.tsx
■ ■ ■ ■ ■ StatsCard.tsx
■ ■ ■ ■ ■ hooks/ # Custom React Hooks
■ ■ ■ ■ ■ useAuth.ts # Auth logic
■ ■ ■ ■ ■ useApi.ts # API calls
■ ■ ■ ■ ■ useSocket.ts # Socket.io
■ ■ ■ ■ ■ useDebounce.ts # Debounce
■ ■ ■ ■ ■ useLocalStorage.ts # Local storage
■ ■ ■ ■ ■ services/ # API Services
■ ■ ■ ■ ■ authService.ts # Auth API calls
■ ■ ■ ■ ■ assessmentService.ts # Assessment API calls
■ ■ ■ ■ ■ competitionService.ts # Competition API calls
■ ■ ■ ■ ■ leaderboardService.ts # Leaderboard API calls
■ ■ ■ ■ ■ analyticsService.ts # Analytics API calls
■ ■ ■ ■ ■ store/ # Redux Store
■ ■ ■ ■ ■ authSlice.ts # Auth state
■ ■ ■ ■ ■ assessmentSlice.ts # Assessment state
■ ■ ■ ■ ■ userSlice.ts # User state
■ ■ ■ ■ ■ store.ts # Store setup
■ ■ ■ ■ ■ styles/ # Global styles
■ ■ ■ ■ ■ globals.css
■ ■ ■ ■ ■ variables.css
■ ■ ■ ■ ■ types/ # TypeScript types
■ ■ ■ ■ ■ user.ts
■ ■ ■ ■ ■ assessment.ts
■ ■ ■ ■ ■ competition.ts
■ ■ ■ ■ ■ api.ts
■ ■ ■ ■ ■ utils/ # Utilities
■ ■ ■ ■ ■ apiClient.ts # Axios instance
■ ■ ■ ■ ■ constants.ts # App constants
■ ■ ■ ■ ■ formatters.ts # Data formatters
■ ■ ■ ■ ■ validators.ts # Client-side validators
■ ■ ■ ■ ■ .env.local.example
■ ■ ■ ■ ■ package.json
■ ■ ■ ■ ■ tsconfig.json
■ ■ ■ ■ ■ tailwind.config.js
■ ■ ■ ■ ■ next.config.js
■
■ ■ ■ ■ ■ docker-compose.yml # Multi-container setup
■ ■ ■ ■ ■ .gitignore
■ ■ ■ ■ ■ README.md # Project documentation
```

Database Models & Schema

User Model

```
Collections: users

{
  _id: ObjectId,
  email: String (unique),
  password: String (hashed),
  firstName: String,
  lastName: String,
  role: String (student, teacher, admin),
  batch: String,
  branch: String,
  profilePicture: String (URL),
  bio: String,
  coinsBalance: Number (default: 500),
  totalCoinsEarned: Number,
  skillAreas: [String],
  isVerified: Boolean,
  createdAt: Date,
  updatedAt: Date
}
```

Assessment Model

```
Collections: assessments

{
  _id: ObjectId,
  title: String,
  topic: String,
  difficulty: String (easy, medium, hard),
  duration: Number (minutes),
  totalQuestions: Number,
  questions: [{
    questionId: ObjectId,
    type: String (mcq, code, debug, design, dev),
    title: String,
    description: String,
    points: Number,
    timeLimit: Number
  }],
  isActive: Boolean,
  createdBy: ObjectId (teacher/admin),
  createdAt: Date
}
```

UserProgress Model

```
Collections: userProgressions

{
  _id: ObjectId,
  userId: ObjectId,
```

```
    topic: String,
    lastAssessmentScore: Number,
    averageScore: Number,
    attemptCount: Number,
    correctAnswers: Number,
    incorrectAnswers: Number,
    currentDifficulty: String,
    status: String (beginner, intermediate, advanced),
    updatedAt: Date
  }
}
```

Competition Model

Collections: competitions

```
{
  _id: ObjectId,
  title: String,
  type: String (individual, group),
  topic: String,
  startDate: Date,
  endDate: Date,
  entryFee: Number (coins),
  maxTeams: Number,
  participants: [ObjectId],
  rounds: [{roundName, questions, duration}],
  status: String (upcoming, active, completed),
  prizePool: {rank1, rank2, rank3},
  createdAt: Date
}
```

API Routes & Endpoints

Endpoint	Method	Purpose	Auth
auth/register	POST	User registration	None
auth/login	POST	User login	None
auth/logout	POST	User logout	JWT
auth/refresh-token	POST	Refresh JWT token	Refresh Token
assessments	GET	Get all assessments	JWT
assessments/:id	GET	Get assessment details	JWT
assessments/:id/submit	POST	Submit assessment	JWT
assessments/:id/questions	GET	Get assessment questions	JWT
competitions	GET	List competitions	JWT
competitions	POST	Create competition (admin)	JWT
competitions/:id/join	POST	Join competition	JWT
competitions/:id/standings	GET	Get standings	JWT
leaderboard/:topic	GET	Get topic leaderboard	JWT
leaderboard/user/:userId	GET	Get user rankings	JWT
skillswap/requests	GET	Get mentorship requests	JWT
skillswap/request	POST	Send mentorship request	JWT
skillswap/accept/:id	PUT	Accept mentorship	JWT
coins/balance	GET	Get coin balance	JWT
coins/transactions	GET	Get coin history	JWT
users/:id	GET	Get user profile	JWT
users/:id	PUT	Update user profile	JWT
analytics/progress	GET	Get progress analytics	JWT

Middleware & Request Validators

Core Middleware

- **authMiddleware.js** - Verify JWT token, extract userId from token, attach user to request
- **roleMiddleware.js** - Check user role (student/teacher/admin), enforce role-based access
- **errorHandler.js** - Catch errors, log, format response, return proper HTTP status codes
- **validateRequest.js** - Run request body/params validation, return 400 if invalid
- **rateLimiter.js** - Limit requests per IP/user to prevent abuse
- **logger.js** - Log all requests with timestamps, method, path, status code

Validation Rules

authValidator.js

```
Register: {
  email: required, valid email, unique
  password: min 8 chars, 1 uppercase, 1 number
  firstName: required, min 2 chars
  lastName: required, min 2 chars
  role: required, enum(student, teacher)
}
```

```
Login: {
  email: required, valid email
  password: required
}
```

assessmentValidator.js

```
Submit Assessment: {
  assessmentId: required, valid ObjectId
  answers: required, array
  timeTaken: required, number
  answers[].questionId: required
  answers[].userAnswer: required
}
```

competitionValidator.js

```
Join Competition: {
  competitionId: required
  teamMembers: array, min 1, max 5
  teamName: required if group comp
}
```

3-Day Sprint Implementation Roadmap

DAY 1: Backend Setup & Authentication

09:00 - 10:30	Project Setup	Initialize Node.js, Express, MongoDB, environment setup, Docker
10:30 - 12:00	Database Models	Create User, Assessment, UserProgress, Competition models
12:00 - 13:00	LUNCH	
13:00 - 14:30	Auth Routes	Register, Login, Logout, Refresh token endpoints
14:30 - 16:00	Auth Middleware	JWT verification, token generation, error handling
16:00 - 16:30	Testing	Test all auth endpoints with Postman
16:30 - 17:00	Documentation	Document auth API with sample requests/responses

DAY 2: Core Features (Assessment, Leaderboard, Coins)

09:00 - 10:30	Assessment API	GET assessments, GET assessment details, GET questions
10:30 - 12:00	Assessment Submission	POST submission, score calculation, progress update
12:00 - 13:00	LUNCH	
13:00 - 14:30	Leaderboard Logic	Calculate rankings, update on submission, leaderboard API
14:30 - 15:30	Coin System	Coin transactions, balance endpoints, reward on completion
15:30 - 16:30	Frontend Setup	Next.js, TypeScript, Tailwind, Redux store, API client
16:30 - 17:00	Testing & Fix	API testing, bug fixes

DAY 3: Frontend UI & Integration

09:00 - 10:30	Auth Pages	Login, Register, OTP verification UI components
10:30 - 12:00	Dashboard Layout	Navbar, Sidebar, dashboard page, responsive design
12:00 - 13:00	LUNCH	
13:00 - 14:30	Assessment UI	Assessment list, quiz page, timer, question display
14:30 - 15:30	Leaderboard & Profile	Leaderboard view, user profile, stats display
15:30 - 16:30	Integration Testing	Connect frontend to backend, test end-to-end flows
16:30 - 17:00	Final Polish	UI fixes, error handling, loading states, deployment prep

3-Day Sprint Deliverables

Backend Deliverables

- ✓ Fully functional Express.js server with MongoDB integration
- ✓ Complete authentication system (register, login, JWT, refresh tokens)
- ✓ Assessment API with question retrieval and submission handling
- ✓ Leaderboard calculation and ranking system
- ✓ Coin economy system with transaction tracking
- ✓ User profile management endpoints
- ✓ All validation middleware and error handling
- ✓ Postman collection with all API endpoints documented
- ✓ Docker setup for containerization

Frontend Deliverables

- ✓ Next.js 14 project with TypeScript setup
- ✓ Tailwind CSS styling with responsive design
- ✓ Redux store with auth and assessment slices
- ✓ Authentication pages (Login, Register, OTP)
- ✓ Dashboard with sidebar navigation
- ✓ Assessment listing and taking interface with timer
- ✓ Leaderboard view with filtering and ranking display
- ✓ User profile page with progress statistics
- ✓ API integration with backend endpoints
- ✓ Loading states and error handling

Testing & Documentation

- ✓ All API endpoints tested and working
- ✓ Frontend-backend integration verified
- ✓ API documentation with Postman/OpenAPI
- ✓ Setup guide for local development
- ✓ Environment configuration examples
- ✓ Database seed scripts for testing
- ✓ GitHub repository with clean commit history

Quick Start Commands

```
Backend:
cd backend
npm install
cp .env.example .env
npm run dev
```

Frontend:

```
cd frontend  
npm install  
npm run dev
```

Docker:

```
docker-compose up -d
```

Success Metrics for 3-Day Sprint

- All core authentication flows working (register, login, token refresh)
- Assessment submission with automatic scoring
- Real-time leaderboard updates
- Responsive UI on desktop and tablet
- 95%+ API endpoint coverage (22+ endpoints)
- Zero critical bugs blocking core workflows
- Clear documentation for future development